

자바 개발자를 위한 Kotlin v2.2.0

Basics

<https://kotlinlang.org/docs/home.html> 공식 문서를 기반으로 작성함

2025-07 조수연

Statement 와 Expression

- **Statement**

- 실행가능한(Executable) 최소의 독립적인 문장.
- 선언문, 할당문, 반복문 등이 포함된다.

- **Expression**

- Statement의 일종
- 평가(Evaluation)를 통해 하나의 "값"이 되는 코드

Statement 와 Expression

- 연어별로 표현식의 범위가 다르다.

- 할당문의 예

- C, Java -> Expression

```
int a = b = c = 0;
```

- Python -> Expression

```
a = b = c = 0;
```

- Kotlin -> Statement

```
var a = b = c = 0 X
```

Coding conventions 요약

<https://kotlinlang.org/docs/coding-conventions.html>

- 파일 확장자는 kt
- 폴더 구조와 패키지 구조를 맞추지 않아도 된다.
- 파일 이름과 클래스 이름이 달라도 된다.
- 한 파일에 두 개 이상의 클래스를 선언할 수 있다.
 - 한 파일에 하나의 클래스만 있을 경우 이름을 맞추는 것을 추천
- 클래스에 소속되지 않은 함수, 변수를 선언할 수 있다.
 - Top level function, Top level variable
- 문장 끝에 세미콜론을 생략한다.

기본 문법

기본 문법

<https://kotlinlang.org/docs/basic-syntax.html>

- package 선언은 코드의 가장 처음
- 코틀린 앱의 진입점은 main 함수 - Top level 함수로 선언한다.

```
fun main() {  
    println("Hello, World!")  
}
```

또는

```
fun main(args: Array<String>) {  
    println(args.contentToString())  
}
```

기본 문법

<https://kotlinlang.org/docs/basic-syntax.html>

- package 선언은 코드의 가장 처음
- 코틀린 앱의 진입점은 main 함수 - Top level 함수로 선언한다.

```
fun main() {  
    println("Hello, World!")  
}
```

또는

```
fun main(args: Array<String>) {  
    println(args.contentToString())  
}
```

기본 문법

<https://kotlinlang.org/docs/basic-syntax.html>

- 표준 입출력
 - `readln()` : 표준 입력으로부터 입력 받은 한 줄의 문자열 반환
 - `print()`, `println()` : 표준 출력으로 문자열 또는 숫자 출력

```
fun main() {  
    val name = readln()  
  
    print(name)  
    println(name)  
}
```

기본 문법

<https://kotlinlang.org/docs/basic-syntax.html>

- 변수

- val : 한 번만 할당 할 수 있는 변수.
- var : 여러번 할당할 수 있는 변수.

```
val message = "Hello"  
val name:String = "World"
```

```
val count = 10  
val age:Int = 0
```

```
var value:Int  
value = readln().toInt()
```

- 변수 이름 뒤에 : Type의 형태로 타입정의
- 할당되는 값으로 추론할 수 있는 경우 타입 생략 가능

타입

<https://kotlinlang.org/docs/basic-types.html>

- 기본 타입 - 기본적으로 모든 타입은 null 을 할당할 수 없는 타입
 - Number 및 각 Number의 unsigned 타입
 - Booleans
 - Characters, Strings
 - Arrays
 - Any
 - Nothing, Unit

타입

<https://kotlinlang.org/docs/basic-types.html>

- Null을 허용하는 타입 - null을 할당하기 위해서는 타입 이름 뒤에 ? 를 추가해야 한다.

```
val v1 = "Hello"  
val v2:String? = "Hello" // 가능  
val v3:String? = null // 가능
```

- null을 허용하지 않는 변수에 null 을 허용하는 변수를 할당할 수 없다.

```
val v1 = "Hello World"  
val v2: String? = "Nice to meet you"  
// val v3: String = v2 // 현재 값이 null이 아니라도 할당할 수 없다.  
val v4: String = v2!! // 일시적으로 Null 을 허용하지 않는 타입으로 변환
```

타입

<https://kotlinlang.org/docs/basic-types.html>

- Safe call 연산자
 - Null pointer Exception이 발생하는 순간은 주로 property 또는 메서드에 접근하는 순간이다.

```
val v1:String? = null // 이 문장 자체가 예외를 발생시키지는 않는다.  
//println(v1.length) // 이 부분이 문제가 된다.  
println(v1?.length) // Safe call 연산자 사용
```

- 객체의 property 또는 메서드 사용 전 ? 로 체크할 수 있다.

타입

<https://kotlinlang.org/docs/basic-types.html>

- Elvis 연산자 ?:
 - 할당할 때 주어진 객체가 null 이라면 다른 값을 사용할 수 있도록 제공한다.

```
val inputLength = str?.length ?: 0
```

Any, Unit

- Any
 - Kotlin 의 최상위 타입. Java에서의 Object에 해당하는 타입
 - Any - null을 허용하지 않는 타입들의 최상위 타입
 - Any? - null 을 허용하는 타입들의 최상위 타입
- Unit
 - Java의 void 에 해당하는 타입
 - 값을 반환하지 않는 함수의 타입

Number

<https://kotlinlang.org/docs/numbers.html>

- 정수형 타입

Type	Size (bits)	Min value	Max value
Byte	8	-128	127
Short	16	-32768	32767
Int	32	-2,147,483,648 (-2^{31})	2,147,483,647 ($2^{31} - 1$)
Long	64	-9,223,372,036,854,775,808 (-2^{63})	9,223,372,036,854,775,807 ($2^{63} - 1$)

- 타입을 명시하지 않는 정수형은 값의 범위가 Int 내면 자동으로 Int로 추론된다.

```
val one = 1 // Int
val threeBillion = 3000000000 // Long
val oneLong = 1L // Long
val oneByte: Byte = 1
```

Number

<https://kotlinlang.org/docs/numbers.html>

- 부동소수점 타입

Type	Size (bits)	Significant bits	Exponent bits	Decimal digits
Float	32	24	8	6-7
Double	64	53	11	15-16

- 타입을 명시 하지 않는 경우 Double로 추론된다.

```
val e = 2.7182818284 // Double
val eFloat = 2.7182818284f // Float, actual value is 2.7182817
```

숫자 끝에 f를 추가하면 Float로 정의된다.

Number

<https://kotlinlang.org/docs/numbers.html>

- 숫자의 표기법
 - 정수 끝에 L : Long
 - 정수를 그냥 적으면: 10진수, 정수 앞에 0x : 16진수, 0과 1 앞에 0b : 2진수
 - 실수에 특별한 표기가 없으면 Double, 실수 끝에 f : Float
- 언더스코어 (underscore : _)를 이용해 가독성을 높일 수 있음

```
val oneMillion = 1_000_000
val creditCardNumber = 1234_5678_9012_3456L
val socialSecurityNumber = 999_99_9999L
val hexBytes = 0xFF_EC_DE_5E
val bytes = 0b11010010_01101001_10010100_10010010
val bigFractional = 1_234_567.7182818284
```

Number

<https://kotlinlang.org/docs/unsigned-integer-types.html>

- Unsigned 숫자를 지워함

Type	Size (bits)	Min value	Max value
UByte	8	0	255
UShort	16	0	65,535
UInt	32	0	4,294,967,295 ($2^{32} - 1$)
ULong	64	0	18,446,744,073,709,551,615 ($2^{64} - 1$)

```
val b: UByte = 1u // UByte, expected type provided
val s: UShort = 1u // UShort, expected type provided
val l: ULong = 1u // ULong, expected type provided

val a1 = 42u // UInt: no expected type provided, constant fits in UInt
val a2 = 0xFFFF_FFFF_FFFFu // ULong: no expected type provided, constant
```

Number

<https://kotlinlang.org/docs/numbers.html>

- 명시적 형변환
 - 자바의 int는 long타입에 할당 가능: 더 큰 범위를 가진 타입으로 자동 형변환

```
int a = 1;  
long b = a; // OK
```

- Kotlin의 숫자 타입들은 더 큰 타입이 없음. 명시적으로 형변환을 해야 함.

```
val a = 1 // Int  
val b: Long = a // Error  
val c: Long = a.toLong()
```

Boolean

<https://kotlinlang.org/docs/booleans.html>

- Boolean
 - true 또는 false
 - Boolean? 타입으로 null 을 저장할 수도 있다.
- Char
 - 한 글자를 저장한다. 작은따옴표 " 로 할당한다.
 - 특수 문자는 백슬래시 \ 로 설정한다.

<https://kotlinlang.org/docs/characters.html>

```
val myTrue: Boolean = true
val myFalse: Boolean = false
val boolNull: Boolean? = null
```

String

<https://kotlinlang.org/docs/strings.html>

- 문자열. 따옴표로 정의한다.
- 여러줄 문장은 따옴표 3개를 사용해 정의할 수 있다.

```
val text = """  
    for (c in "foo")  
        print(c)  
    """
```

- String Template

- \$변수이름 - 표기법으로 동적인 문자열을 생성한다.
- 객체의 속성에 접근하거나 변수 이름 다음 여백이 없을 때. expression이 들어갈 때는 `${}` 로 작성한다.

```
val i = 10  
println("i = $i")
```

```
val s = "abc"  
println("$s.length is ${s.length}")
```

Range & Progressions

<https://kotlinlang.org/docs/ranges.html>

- Collections 중 하나. - iteration이 가능하고 filter, map 등의 기본 연산 모두 가능
- Range: 연속된 값들
 - 1..10 또는 1.rangeTo(10)

```
val range = 1..10  
val range2 = 1.rangeTo(10)
```

- 1..<10 또는 1.rangeUntil(10)

```
val range = 1..<10  
val range2 = 1.rangeUntil(10)
```

Range & Progressions

<https://kotlinlang.org/docs/ranges.html>

- Range 로 무엇을 할 수 있나?

- 랜덤으로 하나 선택하기

```
val range = 1..10
val sample = range.random()
```

- 특정 간격으로 나열하기 (꺼내보기) ==> Progression

```
val range = 1..10
for (i in range) { println(i) }
for (i in range step 2) { println(i) }
```

- first, last, step 등의 속성을 지닌다.

Array

<https://kotlinlang.org/docs/arrays.html>

- 연속된 메모리 공간에 데이터를 저장하는 타입
- 고정 사이즈를 가진다.
- 선언법
 - arrayOf 함수 사용 `var array = arrayOf("1", "2", "3")`
 - Array 클래스의 생성자 사용 (사이즈, 초기화함수)

```
// [0, 0, 0]
val initArray = Array<Int>(3) { 0 }
val indexArray = Array<Int>(3) { it }
// [[0,0],[0,0]]
val twoDArray = Array(2) { Array<Int>(2) { 0 } }
```

Array

<https://kotlinlang.org/docs/arrays.html>

- 지원 메소드
 - sum
 - shuffle
- Collection 으로 변환
 - toList
 - toSet
 - toMap (각 아이템이 Pair타입일 때)

Array

<https://kotlinlang.org/docs/arrays.html>

- 기본적으로 객체 타입을 지원하며 원시 타입은 별도의 클래스를 지원함.

Primitive-type array	Equivalent in Java
<code>BooleanArray</code> ↗	<code>boolean[]</code>
<code>ByteArray</code> ↗	<code>byte[]</code>
<code>CharArray</code> ↗	<code>char[]</code>
<code>DoubleArray</code> ↗	<code>double[]</code>
<code>FloatArray</code> ↗	<code>float[]</code>
<code>IntArray</code> ↗	<code>int[]</code>
<code>LongArray</code> ↗	<code>long[]</code>
<code>ShortArray</code> ↗	<code>short[]</code>

- `Array<Int> ≠ IntArray` : 약간의 동작 차이가 있다. 형변환이 필요하다. 링크 참조

타입 체크와 형변환

<https://kotlinlang.org/docs/typecasts.html>

- 타입 체크 - is, !is

```
if (obj is String) {  
    print(obj.length)  
}  
  
if (obj !is String) { // Same as !(obj is String)  
    print("Not a String")  
} else {  
    print(obj.length)  
}
```

타입 체크와 형변환

<https://kotlinlang.org/docs/typecasts.html>

- Smart cast

```
fun demo(x: Any) {  
    if (x is String) {  
        print(x.length) // x is automatically cast to String  
    }  
}
```

```
if (x !is String) return  
  
print(x.length) // x is automatically cast to String
```

```
// x is automatically cast to String on the right-hand side of `||`  
if (x !is String || x.length == 0) return  
  
// x is automatically cast to String on the right-hand side of `&&`  
if (x is String && x.length > 0) {  
    print(x.length) // x is automatically cast to String  
}
```

타입 체크와 형변환

<https://kotlinlang.org/docs/typecasts.html>

- Unsafe cast -as & Safe cast -as?

```
val x: String = y as String
```

타입 캐스팅이 불가능한 경우 Exception 발생

```
val x: String? = y as? String
```

타입 캐스팅이 불가능한 경우 null 반환

함수의 기초

함수 선언

<https://kotlinlang.org/docs/basic-syntax.html#functions>

- fun 키워드로 선언
- 함수의 return type은 파라미터 정의 뒤에 :Type 형식으로 적는다.

```
fun sum(a: Int, b: Int): Int {  
    return a + b  
}
```

- 반환 값이 없는 경우 타입을 생략하거나 :Unit 으로 표기한다.

```
fun printSum(a: Int, b: Int): Unit {  
    println("sum of $a and $b is ${a + b}")  
}
```

함수 선언

<https://kotlinlang.org/docs/basic-syntax.html#functions>

- 함수의 body 를 expression 으로 작성할 수 있으며 이 경우 반환 타입을 생략할 수 있다.

```
fun sum(a: Int, b: Int) = a + b
```